PHASE 11 OF 18

Multithreading & Concurrency

Processes vs threads, creating threads, lifecycle, sleep/join/interrupt, daemon threads, race conditions, synchronized, volatile, atomics, wait/notify, deadlock, locks, executors, Callable/Future, CompletableFuture, concurrent utilities and virtual threads

IN THIS PHASE

11.1 Multitasking, Processes and Threads
11.2 Creating Threads
11.3 Thread Lifecycle
11.4 Important Thread Methods
11.5 Race Conditions: Why Synchronization Is Needed
11.6 synchronized
11.7 volatile and Visibility
11.8 Atomic Variables
11.9 Inter-Thread Communication: wait(), notify(), notifyAll()
11.10 Deadlock, Livelock and Starvation
11.11 Explicit Locks: ReentrantLock and ReadWriteLock
11.12 The Executor Framework and Thread Pools
11.13 CompletableFuture (Java 8): Asynchronous Pipelines
11.14 Concurrent Collections and Synchronizers
11.15 Virtual Threads (Java 21)
11.16 Best Practices
11.17 Predict the Output
11.18 Interview Questions
11.19 Quick Revision

11.1 Multitasking, Processes and Threads

- **Single tasking:** one task at a time; the CPU idles while waiting for I/O.
- **Multitasking:** several tasks make progress at once. There are two kinds: **process-based** (separate programs, such as a browser and an IDE) and **thread-based** (several threads inside one program).
- **Multithreading:** one Java process runs several **threads** (independent paths of execution) at the same time. For example, a banking server handles thousands of customer requests concurrently, and a UI stays responsive while a report generates in the background.

	Process	Thread
What	An independent running program	A unit of execution **inside** a process
Memory	Own address space	**Shares** the process heap; has its **own stack** and PC register
Creation / context switch	Heavy, slow	Light, fast
Communication	Inter-process communication (sockets, files, pipes)	Shared objects (needs synchronization!)
Failure impact	Isolated	An unhandled error can affect the whole process

> **NOTE**
> **Concurrency** means dealing with many tasks at once (interleaving, even on one core). **Parallelism** means literally executing at the same instant on multiple cores. Java threads give you concurrency, and parallelism when cores are available.

The main thread

Every Java program starts with one thread called **main**, created by the JVM to run `main()`.

```
Thread t = Thread.currentThread();
System.out.println(t.getName());       // main
System.out.println(t.getPriority());   // 5 (NORM_PRIORITY)
System.out.println(t.isDaemon());      // false
System.out.println(t);                 // Thread[#1,main,5,main] (Java 21 format)
```

11.2 Creating Threads

1. Extend Thread

```
class Worker extends Thread {
    @Override
    public void run() {                                    // the task
        System.out.println("Running in " + getName());
    }
}
new Worker().start();                                      // start() creates the new thread
```

2. Implement Runnable (preferred)

```
class Task implements Runnable {
    @Override
    public void run() {
        System.out.println("Running in " + Thread.currentThread().getName());
    }
}
Thread t = new Thread(new Task(), "report-worker");      // task + optional name
t.start();

Thread t2 = new Thread(() -> System.out.println("lambda task"));   // Runnable as a lambda
t2.start();
```

Extending `Thread`	**Implementing** `Runnable`
Uses up the single superclass slot	The class can still extend another class
Task and thread are coupled together	**Task separated** from the thread mechanism
Can't hand the task to a thread pool	Works with `ExecutorService` and thread pools
Each object is a thread	One Runnable can be shared by several threads

3. Implement Callable (returns a result, can throw checked exceptions)

```
Callable<Integer> sumTask = () -> {
    int sum = 0;
    for (int i = 1; i <= 100; i++) sum += i;
    return sum;                                            // returns a value
};
try (ExecutorService pool = Executors.newFixedThreadPool(2)) {   // Java 19+: AutoCloseable
    Future<Integer> f = pool.submit(sumTask);
    System.out.println(f.get());                           // 5050 (blocks until done)
}
```

`Runnable`	`Callable<V>`
`void run()`	`V call() throws Exception`
No result	Returns a result through a `Future<V>`
Can't throw checked exceptions	Can throw checked exceptions
`new Thread(runnable)` or `executor.execute/submit`	`executor.submit(callable)` only (a `Thread` can't take a Callable directly)
Java 1.0	Java 5

start() vs run()

```
Thread t = new Thread(() -> System.out.println("in " + Thread.currentThread().getName()));
t.run();      // in main      <- a plain method call: NO new thread
t.start();    // in Thread-0  <- new thread created; it calls run()
// t.start(); // IllegalThreadStateException: a thread can be started only ONCE
```

> **REMEMBER**
> `start()` asks the JVM to create a new thread, which then calls `run()`. Calling `run()` yourself just runs it on the current thread. A terminated thread can never be restarted: create a new `Thread` or submit the task to a pool again.

11.3 Thread Lifecycle

Java's `Thread.State` enum has **six** states. There is no separate RUNNING state; a thread that is running is in `RUNNABLE`.

```
          start()                          run() ends / uncaught exception
  NEW  ------------>  RUNNABLE  ------------------------------------>  TERMINATED
                    (ready or running)
                      |    ^
  waiting for monitor |    |  lock acquired
  (synchronized)      v    |
                    BLOCKED
                      |    ^
  wait(), join(),     v    |  notify()/notifyAll(), joined thread ends, unpark()
  park()            WAITING
                      |    ^
  sleep(ms),          v    |  time elapses / notify / interrupt
  wait(ms), join(ms) TIMED_WAITING
```

State	Meaning	Entered by
`NEW`	Created, not started	`new Thread(...)`
`RUNNABLE`	Running, or ready and waiting for the CPU (the scheduler decides)	`start()`
`BLOCKED`	Waiting to acquire a monitor lock	Entering a `synchronized` block held by another thread
`WAITING`	Waiting indefinitely for another thread	`wait()`, `join()`, `LockSupport.park()`
`TIMED_WAITING`	Waiting with a timeout	`sleep(ms)`, `wait(ms)`, `join(ms)`
`TERMINATED`	Finished	`run()` returns or throws

```
Thread t = new Thread(() -> {
    try { Thread.sleep(200); } catch (InterruptedException e) { }
});
System.out.println(t.getState());   // NEW
t.start();
Thread.sleep(50);
System.out.println(t.getState());   // TIMED_WAITING (sleeping)
t.join();
System.out.println(t.getState());   // TERMINATED
```

11.4 Important Thread Methods

Method	Static?	Purpose
`start()`	No	Begin execution in a new thread
`run()`	No	The task body (don't call it directly)
`Thread.sleep(ms)`	**Yes**	Pause the current thread (**keeps** any locks held); throws `InterruptedException`
`join()` / `join(ms)`	No	Wait for **that** thread to finish
`Thread.yield()`	**Yes**	Hint that the current thread is willing to give up the CPU (rarely useful)
`interrupt()`	No	Set the target thread's interrupt flag; wakes it from sleep/wait/join
`isInterrupted()` / `Thread.interrupted()`	No / **Yes**	Check the flag / check **and clear** the current thread's flag
`Thread.currentThread()`	**Yes**	The thread executing this code
`getName()` / `setName()`	No	Thread name (helpful in logs and thread dumps)
`getPriority()` / `setPriority(1..10)`	No	Scheduler **hint**: MIN 1, NORM 5, MAX 10
`setDaemon(true)` / `isDaemon()`	No	Mark as a background thread (**before** `start()`)
`isAlive()` / `getState()`	No	Started and not finished? / exact state
`threadId()` (Java 19)	No	Unique id (replaces the deprecated `getId()`)

join: wait for other threads

```
Thread t1 = new Thread(() -> System.out.println("load customers"));
Thread t2 = new Thread(() -> System.out.println("load accounts"));
t1.start(); t2.start();
t1.join(); t2.join();                    // main waits for both
System.out.println("generate report");   // always runs last
```

interrupt: cooperative cancellation

Java has no safe way to *kill* a thread (`stop()` is deprecated and removed). You **request** cancellation with `interrupt()`, and the task checks the flag or reacts to `InterruptedException`.

```
Thread worker = new Thread(() -> {
    while (!Thread.currentThread().isInterrupted()) {       // check the flag
        try {
            System.out.println("polling...");
            Thread.sleep(1000);                              // blocking call
        } catch (InterruptedException e) {
            System.out.println("interrupted, cleaning up");
            Thread.currentThread().interrupt();              // RESTORE the flag
        }
    }
});
worker.start();
Thread.sleep(2500);
worker.interrupt();                                          // politely ask it to stop
```

PITFALL

When `sleep`/`wait`/`join` throws `InterruptedException`, the interrupt flag is **cleared**. Never swallow it with an empty catch. Either propagate it or call `Thread.currentThread().interrupt()` so that code further up knows a cancellation was requested.

Priorities and the scheduler

The **thread scheduler** (part of the JVM and the OS) decides which runnable thread gets the CPU and for how long. Thread execution order is **never guaranteed**, and priorities are only **hints** that many operating systems largely ignore. Never rely on priorities for correctness. `setPriority()` with a value outside 1–10 throws `IllegalArgumentException`.

Daemon threads

User (non-daemon) thread	Daemon thread
The JVM **waits** for it to finish before exiting	The JVM **does not wait**: it is abruptly stopped when the last user thread ends
Application work (main, request handlers)	Background services: GC, monitoring, housekeeping
Default for threads created by `main`	Set with `setDaemon(true)` **before** `start()` (otherwise `IllegalThreadStateException`)

PITFALL

Never use daemon threads for work that must complete (saving data, sending a payment confirmation): their `finally` blocks may never run when the JVM exits.

11.5 Race Conditions: Why Synchronization Is Needed

When several threads **read and write shared mutable data** without coordination, the result depends on timing. This is a **race condition**.

```
class Counter {
    int count = 0;
    void increment() { count++; }          // NOT atomic: read -> add 1 -> write
}

Counter c = new Counter();
Runnable job = () -> { for (int i = 0; i < 10_000; i++) c.increment(); };
Thread t1 = new Thread(job), t2 = new Thread(job);
t1.start(); t2.start(); t1.join(); t2.join();
System.out.println(c.count);               // expected 20000, often less (e.g. 13742; varies)
```

NOTE

On a multi-core machine this usually prints less than 20000, and a different number each run. On a single core (or with a lucky schedule) it may print 20000. That is exactly why race conditions are dangerous: code can pass tests on one machine and fail in production. Correctness must come from synchronization, not from observed output.

```
count = 5
Thread A: read 5 ............ add -> 6 ... write 6
Thread B:        read 5 ...... add -> 6 ............ write 6      => one increment LOST
```

The code that touches shared mutable state, here `count++`, is the **critical section**. It must be executed by only one thread at a time (**mutual exclusion**), and each thread must see the others' latest writes (**visibility**).

11.6 synchronized

Every Java object has an **intrinsic lock** (a **monitor**). A thread must **acquire** the lock to enter a `synchronized` method or block; other threads trying to enter wait in the `BLOCKED` state. The lock is **released automatically** on exit, even if an exception is thrown.

```
class Counter {
    private int count = 0;

    public synchronized void increment() { count++; }   // lock = this
    public synchronized int get() { return count; }     // reads need the lock too (visibility)
}
// With synchronized, the two-thread program above always prints 20000.
```

Synchronized method vs block

```
class BankAccount {
    private double balance;
    private final Object lock = new Object();             // private lock object (best practice)

    public void withdraw(String who, double amount) {
        audit(who, amount);                               // non-critical work: no lock needed
        synchronized (lock) {                             // lock ONLY the critical section
            if (balance >= amount) {
                balance -= amount;
                System.out.println(who + " withdrew " + amount);
            } else {
                System.out.println(who + ": insufficient funds");
            }
        }
    }
    public void deposit(double amount) {
        synchronized (lock) { balance += amount; }
    }
    private void audit(String who, double amt) { /* logging, validation */ }
}
```

Synchronized method	Synchronized block
Locks the whole method	Locks only the critical lines
Lock is `this` (or `Class` for static methods)	Any object you choose: `this`, a private lock object, `X.class`
Simpler	More granular, so better concurrency

Instance lock vs class lock

```
class Demo {
    synchronized void a() { }               // lock: the Demo object (this)
    static synchronized void b() { }        // lock: Demo.class (one per class)
    void c() { synchronized (Demo.class) { } }   // same lock as b()
}
```

Can two threads run these at the same time?	Answer
Two synchronized instance methods on the **same** object	**No** (same `this` lock)
Synchronized instance methods on **different** objects	Yes (different locks)
Two `static synchronized` methods of the same class	**No** (same class lock)
A `static synchronized` and an instance `synchronized` method	**Yes** (different locks)
A synchronized and a **non**-synchronized method on the same object	Yes (the non-synchronized method needs no lock)

> **NOTE**
> Intrinsic locks are **reentrant**: a thread holding a lock can enter another `synchronized` method on the same object without deadlocking itself. Constructors can't be `synchronized` (compile error), and synchronization gives **no ordering or fairness** guarantee.

11.7 volatile and Visibility

Each thread may cache variables (in CPU registers or caches), so one thread's write might **never become visible** to another. `volatile` guarantees that reads and writes go to main memory, and establishes a *happens-before* ordering.

```
class Poller implements Runnable {
    private volatile boolean running = true;     // without volatile, the loop may never stop
    public void run() {
        while (running) { /* poll a queue */ }
        System.out.println("stopped");
    }
    public void stop() { running = false; }      // written by another thread
}
```

Guarantee	volatile	synchronized / locks	Atomic*
Visibility	Yes	Yes	Yes
Atomic single read/write (including long/double)	Yes	Yes	Yes
Atomic compound action (count++, check-then-act)	**No**	Yes	Yes (for that variable)
Blocks other threads	No	Yes	No (lock-free, uses CAS)
Typical use	Status flags, safe publication	Multi-step invariants	Counters, sequence numbers

11.8 Atomic Variables

`java.util.concurrent.atomic` provides **lock-free**, thread-safe operations using CPU compare-and-swap (CAS) instructions.

```
AtomicInteger counter = new AtomicInteger();
counter.incrementAndGet();              // ++count, atomically
counter.addAndGet(5);
counter.compareAndSet(6, 100);          // set to 100 only if it is currently 6 (CAS)
counter.updateAndGet(x -> x * 2);

AtomicLong txnSeq = new AtomicLong(1000);
long nextId = txnSeq.getAndIncrement();  // thread-safe ID generator

LongAdder hits = new LongAdder();        // faster than AtomicLong under heavy contention
hits.increment();
long total = hits.sum();
```

11.9 Inter-Thread Communication: wait(), notify(), notifyAll()

These `Object` methods let threads **coordinate**. A thread waits until a condition becomes true; another thread changes the state and notifies it. They must be called while **holding that object's monitor** (inside `synchronized` on the same object), otherwise `IllegalMonitorStateException` is thrown.

```
class OrderQueue {
    private final Queue<String> queue = new LinkedList<>();
    private final int capacity = 5;

    public synchronized void put(String order) throws InterruptedException {
        while (queue.size() == capacity) wait();     // ALWAYS wait in a while loop
        queue.add(order);
        notifyAll();                                 // wake any waiting consumers
    }

    public synchronized String take() throws InterruptedException {
        while (queue.isEmpty()) wait();              // releases the lock while waiting
        String order = queue.poll();
        notifyAll();                                 // wake any waiting producers
        return order;
    }
}
```

> **REMEMBER**
> Use a `while`, not an `if`, around `wait()`. Threads can wake up spuriously, or another thread may consume the item first, so the condition must be re-checked. Prefer `notifyAll()` unless you are sure exactly one waiter can proceed. In modern code, a `BlockingQueue` (covered later in this phase) does all of this for you.

sleep()	wait()
`Thread` class, **static**	`Object` class, instance method
Keeps locks held	**Releases** that object's monitor while waiting
Pauses for a time	Waits for a condition: notify, timeout or interrupt
Callable anywhere	Only inside `synchronized` on the same object
Wakes after the time elapses	Wakes on `notify`/`notifyAll`, timeout, interrupt or spuriously

11.10 Deadlock, Livelock and Starvation

A **deadlock** happens when two or more threads each hold a lock the other needs, so they wait forever.

```
Object accountA = new Object(), accountB = new Object();

Thread t1 = new Thread(() -> {
    synchronized (accountA) {
        sleepQuietly(100);
        synchronized (accountB) { System.out.println("t1 transfer A->B"); }
    }
});
Thread t2 = new Thread(() -> {
    synchronized (accountB) {                    // opposite order -> DEADLOCK
        sleepQuietly(100);
        synchronized (accountA) { System.out.println("t2 transfer B->A"); }
    }
});
t1.start(); t2.start();                          // both hang forever
```

Four conditions must all hold for a deadlock (the Coffman conditions): mutual exclusion, hold-and-wait, no preemption, and circular wait. Break any one of them to prevent it:

1. **Consistent lock ordering:** always lock accounts in a global order, for example by ascending account ID. This is the most common fix.
2. `tryLock` **with a timeout** (`ReentrantLock`): back off and retry instead of waiting forever.
3. Hold locks for as short a time as possible, and never call unknown code while holding a lock.

```
void transfer(Account from, Account to, BigDecimal amt) {
    Account first  = from.getId() < to.getId() ? from : to;    // global order
    Account second = from.getId() < to.getId() ? to : from;
    synchronized (first) {
        synchronized (second) {
            from.debit(amt);
            to.credit(amt);
        }
    }
}
```

TIP

Diagnose a hung application with a **thread dump**: `jstack <pid>` or `jcmd <pid> Thread.print`. The JVM reports "Found one Java-level deadlock" along with the threads and the locks involved.

Problem	What happens
Deadlock	Threads block forever, each waiting for the other's lock
Livelock	Threads keep reacting to each other, staying busy but making no progress
Starvation	A thread never gets the CPU or a lock because others always win (unfair locks, greedy threads)

11.11 Explicit Locks: ReentrantLock and ReadWriteLock

`java.util.concurrent.locks` offers locks with more features than `synchronized`.

```
private final ReentrantLock lock = new ReentrantLock();     // new ReentrantLock(true) = fair

public void withdraw(double amt) {
    lock.lock();
    try {
        if (balance >= amt) balance -= amt;                 // critical section
    } finally {
        lock.unlock();                                      // ALWAYS unlock in finally
    }
}

public boolean tryTransfer(double amt) throws InterruptedException {
    if (lock.tryLock(500, TimeUnit.MILLISECONDS)) {         // avoid waiting forever
        try { balance -= amt; return true; }
        finally { lock.unlock(); }
    }
    return false;                                           // couldn't get the lock: back off
}

ReadWriteLock rw = new ReentrantReadWriteLock();            // many readers OR one writer
rw.readLock().lock();  try { /* read rates */ }  finally { rw.readLock().unlock(); }
rw.writeLock().lock(); try { /* update rates */ } finally { rw.writeLock().unlock(); }
```

`synchronized`	`ReentrantLock`
Keyword; released automatically	API; **must** call `unlock()` in `finally`
No timeout; can't interrupt a thread waiting for the lock	`tryLock()`, `tryLock(timeout)`, `lockInterruptibly()`
Always unfair	Optional fairness
One wait set (`wait`/`notify`)	Several `Condition` objects (`await`/`signal`)
Simpler, so prefer it for simple cases	Use when you need the extra features

11.12 The Executor Framework and Thread Pools

Creating a new thread per task is expensive, and creating them without limit can crash the JVM. An `ExecutorService` manages a **pool** of reusable worker threads and a **queue** of tasks. You submit *tasks*; the pool decides *which thread* runs them.

```
  submit(task) --> [  Task Queue  ] --> Worker-1  (reused)
  submit(task) -->  t5 t4 t3 t2 t1  --> Worker-2  (reused)
  submit(task) -->                  --> Worker-3  (reused)
```

Factory (`Executors.`)	Behaviour	Use for
`newFixedThreadPool(n)`	n threads, unbounded queue	Steady CPU-bound or limited-I/O work
`newSingleThreadExecutor()`	One thread; tasks run sequentially in order	Ordered background processing
`newCachedThreadPool()`	Creates threads as needed, reuses idle ones (60 s)	Many short tasks (can grow without limit!)
`newScheduledThreadPool(n)`	Delayed or periodic tasks	Polling, report schedules, timeouts
`newWorkStealingPool()`	`ForkJoinPool` with parallelism = cores	Divide-and-conquer parallel tasks
`newVirtualThreadPerTaskExecutor()` (Java 21)	One **virtual** thread per task	High-concurrency blocking I/O

```
ExecutorService pool = Executors.newFixedThreadPool(3);
for (int i = 1; i <= 6; i++) {
    int id = i;                                              // effectively final copy
    pool.execute(() -> System.out.println("Task " + id + " on "
            + Thread.currentThread().getName()));            // pool-1-thread-1..3 reused
}
pool.shutdown();                                             // stop accepting new tasks
if (!pool.awaitTermination(60, TimeUnit.SECONDS)) {          // wait for running tasks
    pool.shutdownNow();                                      // then interrupt stragglers
}

ScheduledExecutorService ses = Executors.newScheduledThreadPool(1);
ses.scheduleAtFixedRate(() -> System.out.println("heartbeat"), 0, 5, TimeUnit.SECONDS);
```

Method	Behaviour
`execute(Runnable)`	Fire and forget; no result; exceptions go to the thread's handler
`submit(Runnable / Callable)`	Returns a `Future`; exceptions are captured and rethrown from `get()` as `ExecutionException`
`invokeAll(tasks)`	Runs all tasks, returns a list of Futures when **all** are done
`invokeAny(tasks)`	Returns the result of the **first** task to succeed; cancels the rest
`shutdown()`	No new tasks; queued and running tasks finish; doesn't block
`shutdownNow()`	Interrupts running tasks; returns tasks never started
`awaitTermination(t, unit)`	Blocks until all tasks finish or the timeout passes
`close()` (Java 19+)	Shutdown + await, so an executor can be used in try-with-resources

Future

```
ExecutorService pool = Executors.newFixedThreadPool(2);
Future<BigDecimal> balance = pool.submit(() -> fetchBalance("SB-1001"));   // Callable
// ... do other work meanwhile ...
BigDecimal b = balance.get(2, TimeUnit.SECONDS);   // blocks; TimeoutException if too slow
balance.isDone();  balance.cancel(true);  balance.isCancelled();
```

ThreadPoolExecutor under the hood

```
ThreadPoolExecutor exec = new ThreadPoolExecutor(
        4,                                  // corePoolSize: threads kept alive
        8,                                  // maximumPoolSize: used when the queue is full
        30, TimeUnit.SECONDS,               // keepAlive for threads above core
        new ArrayBlockingQueue<>(100),      // BOUNDED queue (backpressure)
        new ThreadPoolExecutor.CallerRunsPolicy());   // what to do when saturated
```

Rejection policies are `AbortPolicy` (the default: throws `RejectedExecutionException`), `CallerRunsPolicy` (the caller runs the task, which slows producers), `DiscardPolicy` and `DiscardOldestPolicy`.

> **PITFALL**
> `newFixedThreadPool` uses an **unbounded** queue, and `newCachedThreadPool` an **unbounded number of threads**. Under heavy load they can cause `OutOfMemoryError`. Production code often builds a `ThreadPoolExecutor` with bounded queues, named threads and a deliberate rejection policy. Also: **always shut down** executors, because their non-daemon threads keep the JVM alive.

11.13 CompletableFuture (Java 8): Asynchronous Pipelines

`Future.get()` blocks. `CompletableFuture` lets you **chain**, **combine** and **handle errors** asynchronously, much like promises in JavaScript.

```
CompletableFuture<BigDecimal> balanceF =
        CompletableFuture.supplyAsync(() -> fetchBalance("SB-1001"));   // runs in a pool
CompletableFuture<BigDecimal> rateF =
        CompletableFuture.supplyAsync(() -> fetchInterestRate("SAVINGS"));

CompletableFuture<String> report = balanceF
        .thenCombine(rateF, (bal, rate) -> bal.multiply(rate))          // combine two results
        .thenApply(interest -> "Projected interest: " + interest)       // transform
        .exceptionally(ex -> "Service unavailable: " + ex.getMessage()) // fallback on error
        .orTimeout(3, TimeUnit.SECONDS);                                 // Java 9

System.out.println(report.join());          // join() = get() without checked exceptions

CompletableFuture<Void> all = CompletableFuture.allOf(balanceF, rateF);  // wait for all
CompletableFuture<Object> any = CompletableFuture.anyOf(balanceF, rateF); // first to finish
```

Method	Purpose
`supplyAsync(supplier)` / `runAsync(runnable)`	Start async work (common `ForkJoinPool`, or a pool you pass in)
`thenApply(fn)`	Transform the result (like `map`)
`thenCompose(fn)`	Chain another async call (like `flatMap`)
`thenCombine(other, fn)`	Merge two independent results
`thenAccept(consumer)` / `thenRun(r)`	Consume the result / run an action afterwards
`exceptionally(fn)` / `handle(fn)`	Recover from errors / handle the result or error
`allOf(...)` / `anyOf(...)`	Wait for all / for the first
`join()` / `get()`	Block for the result (unchecked / checked exceptions)

11.14 Concurrent Collections and Synchronizers

Utility	What it does	Typical use
`ConcurrentHashMap`	Thread-safe map with fine-grained locking; atomic `merge`, `compute`, `putIfAbsent`	Shared caches, counters by key (internals in Phase 14)
`CopyOnWriteArrayList`	Copies the array on every write; lock-free reads; iterators never throw CME	Listener lists: read often, written rarely
`BlockingQueue` (`ArrayBlockingQueue`, `LinkedBlockingQueue`)	`put` blocks when full; `take` blocks when empty	Producer–consumer pipelines
`CountDownLatch(n)`	Threads `await()` until `countDown()` has been called n times (one-shot)	Wait for n services to start, or n tasks to finish
`CyclicBarrier(n)`	n threads wait for each other, then proceed together (reusable)	Phased parallel computations
`Semaphore(n)`	At most n permits held at once	Limit concurrent DB/API calls (throttling)
`ThreadLocal<T>`	A separate copy of a variable per thread	Per-request context, non-thread-safe formatters

```
BlockingQueue<String> queue = new ArrayBlockingQueue<>(10);
Thread producer = new Thread(() -> {
    try { for (int i = 1; i <= 5; i++) queue.put("TXN-" + i); }
    catch (InterruptedException e) { Thread.currentThread().interrupt(); }
});
Thread consumer = new Thread(() -> {
    try { for (int i = 1; i <= 5; i++) System.out.println("processed " + queue.take()); }
    catch (InterruptedException e) { Thread.currentThread().interrupt(); }
});
producer.start(); consumer.start();       // no wait/notify code needed

CountDownLatch ready = new CountDownLatch(3);
for (String svc : List.of("db", "cache", "mq")) {
    new Thread(() -> { System.out.println(svc + " up"); ready.countDown(); }).start();
}
ready.await();                            // main continues only after all 3 count down
System.out.println("all services ready");

Semaphore apiLimit = new Semaphore(5);    // max 5 concurrent calls to a partner API
apiLimit.acquire();
try { /* call API */ } finally { apiLimit.release(); }
```

PITFALL

With thread pools, a `ThreadLocal` value **stays on the reused thread** and can leak into the next request (or leak memory). Always call `threadLocal.remove()` in a `finally` block.

11.15 Virtual Threads (Java 21)

Classic ("platform") threads map one-to-one to OS threads, which are expensive (around 1 MB of stack each), so servers are limited to a few thousand of them. **Virtual threads** are lightweight threads managed by the JVM. Millions can exist, and when one blocks on I/O, the JVM **unmounts** it from its carrier OS thread, so that thread is freed for other work.

```
Thread vt = Thread.ofVirtual().start(() -> System.out.println("hello from a virtual thread"));
vt.join();
Thread.startVirtualThread(() -> callPaymentApi());

try (ExecutorService exec = Executors.newVirtualThreadPerTaskExecutor()) {
    for (int i = 0; i < 10_000; i++) {
        int id = i;
        exec.submit(() -> {                // 10,000 concurrent blocking calls: no problem
            Thread.sleep(Duration.ofSeconds(1));
            return id;
        });
    }
}                                          // close() waits for all tasks
```

	Platform thread	Virtual thread
Backed by	One OS thread each	A small pool of carrier OS threads
Cost	Heavy (stack memory, OS scheduling)	Very cheap: create one per task
Good for	CPU-bound work	**Blocking I/O-bound** work (HTTP, DB, file)
Pooling	Yes (pools exist to reuse threads)	**Don't pool**: create a new one per task

NOTE

Virtual threads make simple blocking code (thread-per-request) scale like reactive code. Spring Boot 3.2+ can enable them with `spring.threads.virtual.enabled=true`. On Java 21–23, blocking inside `synchronized` *pins* the carrier thread (use `ReentrantLock` for long blocking sections); Java 24 (JEP 491) removed most pinning. Virtual threads don't make CPU-bound work faster.

11.16 Best Practices

1. Prefer **executors** (or virtual threads) over creating `Thread`s manually; always shut executors down.
2. **Minimise shared mutable state**: use immutable objects, confinement (local variables) and message passing (queues).
3. Keep critical sections **small**; lock on **private final** lock objects, not `this` or Strings.
4. Use **higher-level utilities** (`ConcurrentHashMap`, atomics, `BlockingQueue`, latches) before `wait`/`notify`.
5. Acquire multiple locks in a **consistent global order**; consider `tryLock` with timeouts.
6. Always `unlock()` in `finally`; use `while` loops around waits.
7. Never swallow `InterruptedException`: restore the flag or propagate it.
8. **Name your threads** (a `ThreadFactory`), so logs and thread dumps are readable.
9. Don't rely on `sleep()` for synchronization in code or **tests**. Use latches, `join` or `Awaitility`-style polling in tests.
10. Bound queues and pools in production, and choose a rejection policy deliberately.

11.17 Predict the Output

```
Thread t = new Thread(() -> System.out.print(Thread.currentThread().getName() + " "));
t.run();                                          // main   (no new thread)
t.start(); t.join();                              // Thread-0
System.out.println();

Thread d = new Thread(() -> { });
d.start();
try { d.setDaemon(true); }
catch (IllegalThreadStateException e) { System.out.println("ITSE"); }   // ITSE

Object lock = new Object();
try { lock.wait(); }
catch (IllegalMonitorStateException e) { System.out.println("IMSE"); }  // IMSE

AtomicInteger ai = new AtomicInteger(5);
System.out.println(ai.getAndIncrement() + " " + ai.incrementAndGet());  // 5 7
System.out.println(ai.compareAndSet(7, 10) + " " + ai.get());          // true 10

Thread n = new Thread(() -> { });
System.out.println(n.getState());                 // NEW
n.start(); n.join();
System.out.println(n.getState() + " " + n.isAlive());   // TERMINATED false
```

11.18 Interview Questions

Q. What is the difference between a process and a thread?

A process is an independent program with its own memory. A thread is a lightweight execution path inside a process that shares the heap with other threads but has its own stack and program counter.

Q. How do you create a thread? Which way is preferred?

Extend `Thread`, implement `Runnable` (or use a lambda), or submit a `Callable` to an executor. Implementing `Runnable`/`Callable` is preferred: it keeps inheritance free, separates the task from the thread, and works with thread pools.

Q. What happens if you call `run()` instead of `start()`? Can a thread be started twice?

`run()` executes on the current thread, so no new thread is created. Calling `start()` twice throws `IllegalThreadStateException`.

Q. What are the thread states?

NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING and TERMINATED. There is no RUNNING state in `Thread.State`; running threads are RUNNABLE.

Q. What is a race condition, and how do you prevent it?

Incorrect results caused by unsynchronized concurrent access to shared mutable data, where the outcome depends on timing. Prevent it with `synchronized`, locks, atomic classes, concurrent collections, or by avoiding shared mutable state.

Q. What is the difference between `synchronized` and `volatile`?

`synchronized` provides mutual exclusion **and** visibility for a block of code. `volatile` provides only visibility and ordering for a single variable; it doesn't make compound actions like `count++` atomic.

Q. What is the difference between `sleep()` and `wait()`?

`sleep` is a static Thread method that pauses for a time and keeps locks. `wait` is an Object method, must be called holding that object's monitor, releases it, and waits for notify, a timeout or an interrupt.

Q. Why must `wait()` be called in a loop?

Because of spurious wakeups, and because the condition may have changed again by the time the thread reacquires the lock. The condition must be re-checked.

Q. What is a deadlock, and how do you avoid it?
Threads waiting forever for locks held by each other. Avoid it with consistent lock ordering, `tryLock` timeouts, minimal lock scope, and by not calling external code while holding locks. Detect it with thread dumps (`jstack`).

Q. What are the advantages of `ReentrantLock` over `synchronized`?
Timed and interruptible lock attempts (`tryLock`), optional fairness, and multiple `Condition`s. The cost is that unlocking is manual and must go in `finally`.

Q. Why use thread pools? Explain `shutdown` vs `shutdownNow`.
Pools reuse threads, limit concurrency and queue tasks, which is cheaper and safer than creating threads per task. `shutdown` stops new submissions and lets queued tasks finish. `shutdownNow` interrupts running tasks and returns the tasks that never started.

Q. What is the difference between Runnable and Callable, and what is a Future?
Callable returns a value and can throw checked exceptions; Runnable can't. A `Future` represents a pending result: `get()` blocks, and you can also check `isDone()` or `cancel()`.

Q. How is CompletableFuture better than Future?
It supports non-blocking chaining (`thenApply`, `thenCompose`), combining results (`thenCombine`, `allOf`), error recovery (`exceptionally`) and timeouts, without blocking threads at every step.

Q. What are daemon threads?
Background threads that don't keep the JVM alive. The JVM exits when only daemon threads remain. Mark a thread with `setDaemon(true)` before `start()`.

Q. What are virtual threads, and when should you use them?
Lightweight JVM-managed threads (Java 21) that are cheap enough to create one per task. They are ideal for high-concurrency, blocking I/O workloads (web requests, DB and API calls), not for CPU-bound work, and they shouldn't be pooled.

11.19 Quick Revision

- Threads share the **heap**, but each has its own **stack**. Execution order is never guaranteed; priorities are hints.
- Create: `Runnable`/lambda (preferred), `extends Thread`, `Callable` via an executor. `start()` once; `run()` directly creates no thread.
- States: NEW → RUNNABLE ↔ BLOCKED / WAITING / TIMED_WAITING → TERMINATED.
- `sleep` keeps locks; `wait` releases them (needs `synchronized`; use a `while` loop). `join` waits for a thread to finish.
- Interrupt = cooperative cancellation. Restore the flag after catching `InterruptedException`.
- Daemon threads don't keep the JVM alive; call `setDaemon` before `start`.
- Race condition: `count++` isn't atomic. Fix with `synchronized`, locks or atomics.
- `synchronized` instance method → lock `this`; static → lock `Class`. Locks are reentrant; release is automatic.
- `volatile` = visibility only (flags); atomics = lock-free counters (CAS); `LongAdder` for high contention.
- Deadlock: needs all four Coffman conditions. Prevent with lock ordering or `tryLock`; diagnose with `jstack`.
- `ReentrantLock`: `lock()` / `try` / `finally unlock()`; `tryLock`, fairness, Conditions. `ReadWriteLock` for read-heavy data.
- Executors: fixed, single, cached, scheduled, virtual-per-task. `execute` vs `submit` (Future). Always shut down; bound queues in production.
- `CompletableFuture`: `supplyAsync` → `thenApply`/`thenCompose`/`thenCombine` → `exceptionally` → `join`.
- Utilities: `ConcurrentHashMap`, `CopyOnWriteArrayList`, `BlockingQueue`, `CountDownLatch`, `CyclicBarrier`, `Semaphore`, `ThreadLocal` (call `remove()`).

- Virtual threads (Java 21): a million cheap threads for blocking I/O; don't pool them.